

5.1 Introduction

The flash accelerator block in the LPC176x/5x allows maximization of the performance of the Cortex-M3 processor when it is running code from flash memory, while also saving power. The flash accelerator also provides speed and power improvements for data accesses to the flash memory.

5.2 Flash accelerator blocks

The flash accelerator is divided into several functional blocks:

- AHB-Lite bus interface, accessible by the Cortex-M3 I-code and D-code buses, as well as by the General Purpose DMA Controller
- An array of eight 128-bit buffers
- Flash accelerator control logic, including address compare and flash control
- A flash memory interface

[Figure 13](#) shows a simplified diagram of the flash accelerator blocks and data paths.

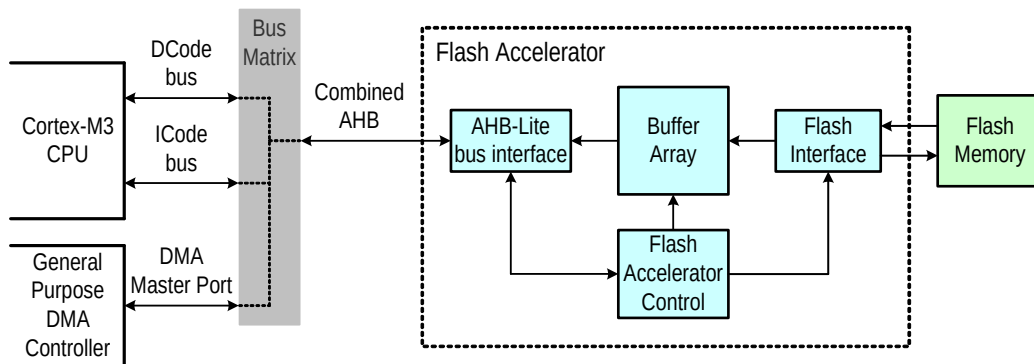


Fig 13. Simplified block diagram of the flash accelerator showing potential bus connections

In the following descriptions, the term “fetch” applies to an explicit flash read request from the CPU. “Prefetch” is used to denote a flash read of instructions beyond the current processor fetch address.

5.2.1 Flash memory bank

There is one bank of flash memory controlled by the LPC176x/5x flash accelerator.

Flash programming operations are not controlled by the flash accelerator, but are handled as a separate function. A Boot ROM contains flash programming algorithms that may be called as part of the application program, and a loader that may be run to allow programming of the flash memory.

5.2.2 Flash programming Issues

Since the flash memory does not allow accesses during programming and erase operations, it is necessary for the flash accelerator to force the CPU to wait if a memory access to a flash address is requested while the flash memory is busy with a programming operation. Under some conditions, this delay could result in a Watchdog time-out. The user will need to be aware of this possibility and take steps to insure that an unwanted Watchdog reset does not cause a system failure while programming or erasing the flash memory.

In order to preclude the possibility of stale data being read from the flash memory, the LPC176x/5x flash accelerator buffers are automatically invalidated at the beginning of any flash programming or erase operation. Any subsequent read from a flash address will cause a new fetch to be initiated after the flash operation has completed.

5.3 Register description

The flash accelerator is controlled by the register shown in [Table 48](#). More detailed descriptions follow.

Table 48. Summary of flash accelerator registers

Name	Description	Access	Reset value ^[1]	Address
FLASHCFG	Flash Accelerator Configuration Register. Controls flash access timing. See Table 49 .	R/W	0x303A	0x400F C000

[1] Reset Value reflects the data stored in defined bits only. It does not include reserved bits content.

5.4 Flash Accelerator Configuration register (FLASHCFG - 0x400F C000)

Configuration bits select the flash access time, as shown in [Table 49](#). **The lower bits of FLASHCFG control internal flash accelerator functions and should not be altered.**

Following reset, flash accelerator functions are enabled and flash access timing is set to a default value of 4 clocks.

Changing the FLASHCFG register value causes the flash accelerator to invalidate all of the holding latches, resulting in new reads of flash information as required. This guarantees synchronization of the flash accelerator to CPU operation.

Table 49. Flash Accelerator Configuration register (FLASHCFG - address 0x400F C000) bit description

Bit	Symbol	Value	Description	Reset value
11:0	-	-	Reserved, user software should not change these bits from the reset value.	0x03A
15:12	FLASHTIM		Flash access time. The value of this field plus 1 gives the number of CPU clocks used for a flash access. Warning: improper setting of this value may result in incorrect operation of the device.	0x3
		0000	Flash accesses use 1 CPU clock. Use for up to 20 MHz CPU clock.	
		0001	Flash accesses use 2 CPU clocks. Use for up to 40 MHz CPU clock.	
		0010	Flash accesses use 3 CPU clocks. Use for up to 60 MHz CPU clock.	
		0011	Flash accesses use 4 CPU clocks. Use for up to 80 MHz CPU clock.	
		0100	Flash accesses use 5 CPU clocks. Use for up to 100 MHz CPU clock. Use for up to 120 Mhz for LPC1759 and LPC1769 only.	
		0101	Flash accesses use 6 CPU clocks. This "safe" setting will work under any conditions.	
		Other	Intended for potential future higher speed devices.	
31:16	-		Reserved. The value read from a reserved bit is not defined.	NA

5.5 Operation

Simply put, the flash accelerator attempts to have the next Cortex-M3 instruction that will be needed in its latches in time to prevent CPU fetch stalls. The LPC176x/5x uses one bank of flash memory. The flash accelerator includes an array of eight 128-bit buffers to store both instructions and data in a configurable manner. Each 128-bit buffer in the array can include four 32-bit instructions, eight 16-bit instructions or some combination of the two. During sequential code execution, a buffer typically contains the current instruction and the entire flash line that contains that instruction, or one flash line of data containing a previously requested address. Buffers are marked according to how they are used (as instruction or data buffers), and when they have been accessed. This information is used to carry out the buffer replacement strategy.

The Cortex-M3 provides a separate bus for instruction access (I-code) and data access (D-code) in the code memory space. These buses, plus the General Purpose DMA Controllers's master port, are arbitrated by the AHB multilayer matrix. Any access to the flash memory's address space is presented to the flash accelerator.

If a flash instruction fetch and a flash data access from the CPU occur at the same time, the multilayer matrix gives precedence to the data access. This is because a stalled data access always slows down execution, while a stalled instruction fetch often does not. When the flash data access is concluded, any flash fetch or prefetch that had been in progress is re-initiated.

Branches and other program flow changes cause a break in the sequential flow of instruction fetches described above. Buffer replacement strategy in the flash accelerator attempts to maximize the chances that potentially reusable information is retained until it is needed again.

If an attempt is made to write directly to the flash memory without using the normal flash programming interface (via Boot ROM function calls), the flash accelerator generates an error condition. The CPU treats this error as a data abort. The GPDMA handles error conditions as described in [Section 31.4.1.6.3](#).

When an Instruction Fetch is not satisfied by existing contents of the buffer array, nor has a prefetch been initiated for that flash line, the CPU will be stalled while a fetch is initiated for the related 128-bit flash line. If a prefetch has been initiated but not yet completed, the CPU is stalled for a shorter time since the required flash access is already in progress.

Typically, a flash prefetch is begun whenever an access is made to a just prefetched address, or to a buffer whose immediate successor is not already in another buffer. A prefetch in progress may be aborted by a data access, in order to minimize CPU stalls.

A prefetched flash line is latched within the flash memory, but the flash accelerator does not capture the line in a buffer until the CPU presents an address that is contained within the prefetched flash line. If the core presents an instruction address that is not already buffered and is not contained in the prefetched flash line, the prefetched line will be discarded.

Some special cases include the possibility that the CPU will request a data access to an address already contained in an instruction buffer. In this case, the data will be read from the buffer as if it was a data buffer. The reverse case, if the CPU requests an instruction address that can be satisfied from an existing data buffer, causes the instruction to be supplied from the data buffer, and the buffer to be changed into an instruction buffer. This causes the buffer to be handled differently when the flash accelerator is determining which buffer is to be overwritten next.